

Exp No: 24

MODELING CLOCK-DRIVEN CONCURRENT PROCESSES USING SC_THREAD AND WAIT() IN SYSTEMC

```
GNU nano 8.7.1 test_systemc.cpp
#include <systemc.h>

SC_MODULE(ClockExample) {
    sc_in<bool> clk;

    void process() {
        while (true) {
            wait();
            std::cout << "Time: " << sc_time_stamp()
            << " - Clock Triggered" << std::endl;
        }
    }

    SC_CTOR(ClockExample) {
        SC_THREAD(process);
        sensitive << clk.pos();
    }
};

int sc_main(int argc, char* argv[]) {
    sc_clock clock("clock", 1, SC_NS);

    ClockExample module("module");
    module.clk(clock);

    sc_start(5, SC_NS);

    return 0;
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo      M-A Set Mark  M-J To Bracket
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^_ Justify    ^/ Go To Line M-E Redo      M-C Copy      ^B Where Was
```

```
anish@DESKTOP-BUFCRBD:~$ nano test_systemc.cpp
anish@DESKTOP-BUFCRBD:~$ g++ -o test_systemc test_systemc.cpp -lsystemc
anish@DESKTOP-BUFCRBD:~$ ./test_systemc

SystemC 3.0.2-Accellera --- Nov  5 2025 12:44:46
Copyright (c) 1996-2025 by all Contributors,
ALL RIGHTS RESERVED
Time: 0 s - Clock Triggered
Time: 1 ns - Clock Triggered
Time: 2 ns - Clock Triggered
Time: 3 ns - Clock Triggered
Time: 4 ns - Clock Triggered
anish@DESKTOP-BUFCRBD:~$
```